

Lista 1 - JCRCRIP

Andrey Rodrigues - JC3033872

github: <https://github.com/AndreyRodr/Introducao-a-criptografia/tree/main>

1. Um aplicativo cifra mensagens, mas não autentica o remetente. Quais propriedades ele oferece e quais deixa em aberto?

R: Ele oferece confidencialidade, pois a operação de cifração limita quem pode compreender os dados. Ele deixa em aberto a autenticidade, pois não verifica a identidade alegada ou a origem da mensagem, também deixa em aberto a integridade, já que apenas cifrar não detecta necessariamente se o arquivo foi trocado por outro arquivo cifrado.

2. Classifique como ataque passivo ou ativo:

- (a) copiar tráfego: Ataque passivo
- (b) trocar o valor de uma transferência: Ataque Ativo
- (c) medir horários e tamanhos: Ataque passivo
- (d) repetir uma requisição antiga: Ataque Ativo

3. Construa um modelo de ameaça curto para notas armazenadas em um sistema escolar.

R: Ativos: O principal ativo que possui valor e precisa de proteção são as notas dos alunos registradas no sistema.

Adversários e Capacidades: Uma conta de aluno tentando visualizar informações de outras turmas, ou adversários tentando adulterar informações no banco de dados.

Objetivos: Impedir a leitura não autorizada de notas, garantindo a confidencialidade. Além disso, registrar toda alteração feita por docentes, garantindo auditoria e integridade.

4. Explique o princípio de Kerckhoffs com suas palavras.

R: A segurança de um sistema não deve depender de algoritmos escondidos, o sistema deve ser seguro mesmo quando o projeto é totalmente público, as keys utilizadas no projeto devem ser as únicas coisas que ficam em segredo para garantir a proteção.

5. Por que uma cópia de segurança não substitui integridade criptográfica?

R: Pois o backup é utilizado após um evento de perda/corrupção, porém ele não é capaz de detectar se houve alguma alteração não permitida nos dados.

6. Se todos usam a mesma chave, quem consegue provar qual pessoa enviou a mensagem?

R: Se todos usam a mesma chave compartilhada, ninguém consegue provar qual pessoa específica enviou a mensagem.

7. O que um atacante aprende apenas observando tamanhos e horários?

R: O atacante pode ver quem conversou com quem, quando e por quanto tempo durou a interação também. Ele aprende informações sobre o contexto da comunicação mesmo que o conteúdo permaneça oculto pela confidencialidade.

8. Qual é o ativo mais importante no sistema escolar proposto?

R: As notas dos alunos, proteger as notas de acessos indevidos e garantir o registro de suas alterações é o mais importante pro sistema.

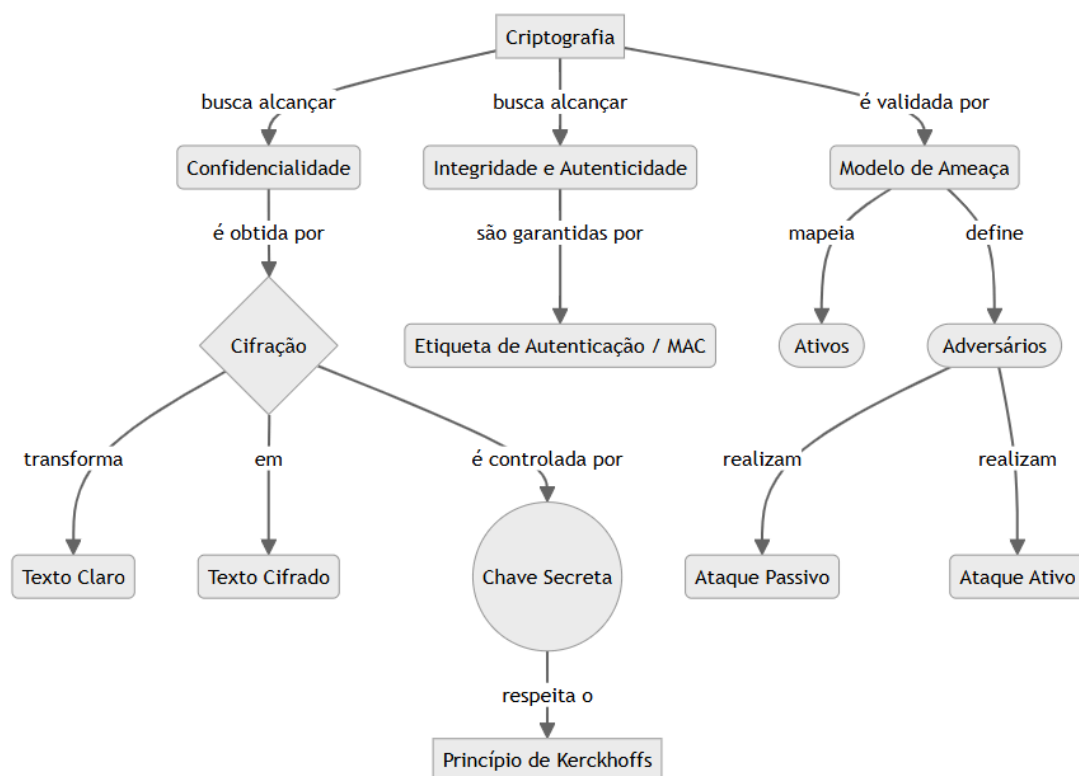
9. Analise a confusão frequente a seguir e escreva uma orientação correta: Tratar codificação Base64 como criptografia. Ela é reversível sem chave.

R: Codificações operam segundo uma regra pública voltada para converter a representação dos dados, sem nenhum objetivo de proporcionar sigilo. Como ela é perfeitamente reversível sem a necessidade de nenhuma chave secreta, tratá-la como criptografia é uma armadilha, pois não oferece a propriedade de confidencialidade contra invasores

10. Construa um mapa conceitual que relacione os conceitos essenciais deste encontro. Inclua pelo menos seis ligações explicadas.

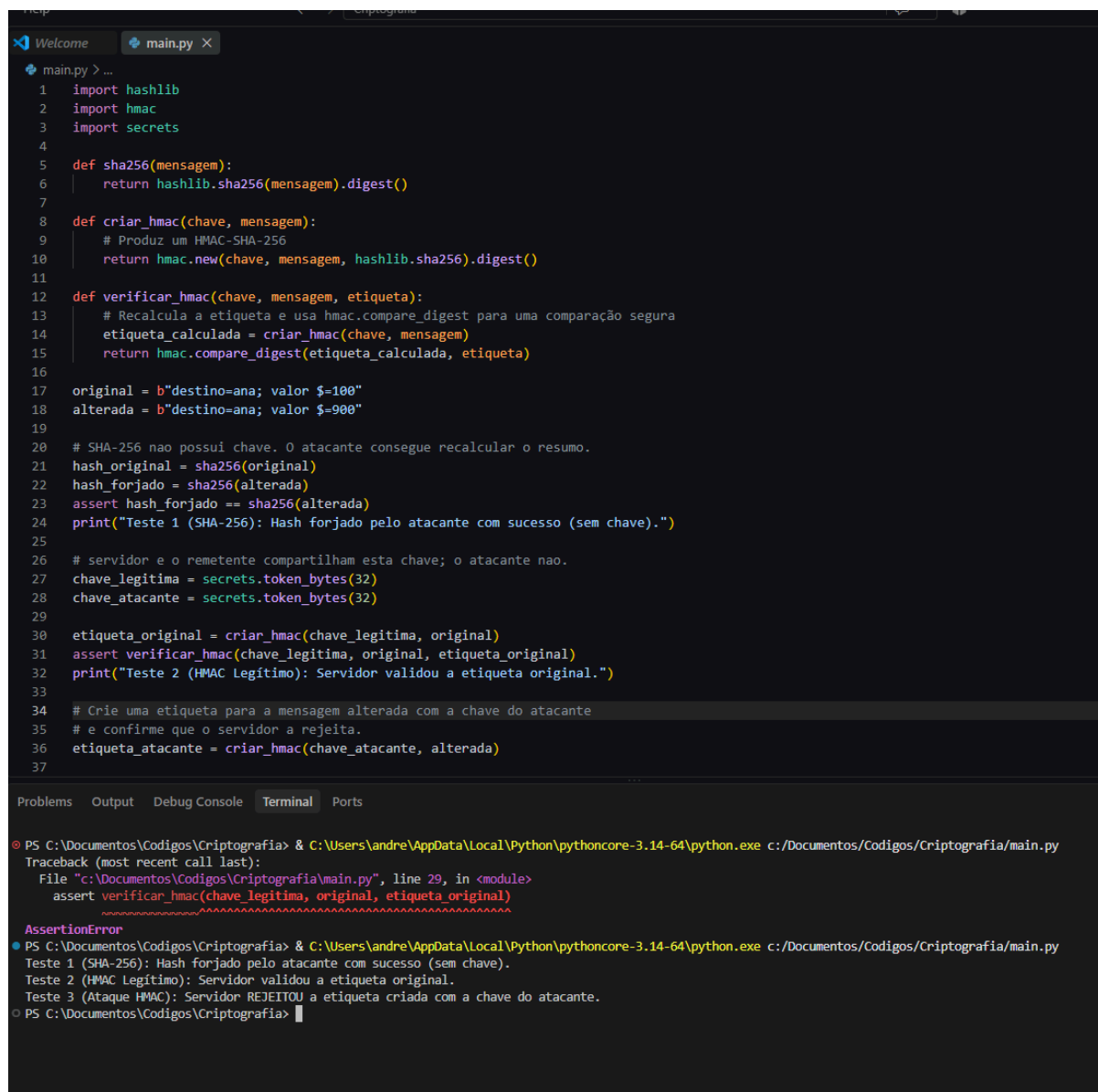
Mapa Conceitual: Introdução à Criptografia

Relação dos conceitos essenciais do Encontro 01



Atividade 11 — Prática de programação

1. Saídas



```
1 import hashlib
2 import hmac
3 import secrets
4
5 def sha256(mensagem):
6     return hashlib.sha256(mensagem).digest()
7
8 def criar_hmac(chave, mensagem):
9     # Produz um HMAC-SHA-256
10    return hmac.new(chave, mensagem, hashlib.sha256).digest()
11
12 def verificar_hmac(chave, mensagem, etiqueta):
13     # Recalcula a etiqueta e usa hmac.compare_digest para uma comparação segura
14     etiqueta_calculada = criar_hmac(chave, mensagem)
15     return hmac.compare_digest(etiqueta_calculada, etiqueta)
16
17 original = b"destino=ana; valor $=100"
18 alterada = b"destino=ana; valor $=900"
19
20 # SHA-256 nao possui chave. O atacante consegue recalculer o resumo.
21 hash_original = sha256(original)
22 hash_forjado = sha256(alterada)
23 assert hash_forjado == sha256(alterada)
24 print("Teste 1 (SHA-256): Hash forjado pelo atacante com sucesso (sem chave).")
25
26 # servidor e o remetente compartilham esta chave; o atacante nao.
27 chave_legitima = secrets.token_bytes(32)
28 chave_atacante = secrets.token_bytes(32)
29
30 etiqueta_original = criar_hmac(chave_legitima, original)
31 assert verificar_hmac(chave_legitima, original, etiqueta_original)
32 print("Teste 2 (HMAC Legitimo): Servidor validou a etiqueta original.")
33
34 # Crie uma etiqueta para a mensagem alterada com a chave do atacante
35 # e confirme que o servidor a rejeita.
36 etiqueta_atacante = criar_hmac(chave_atacante, alterada)
37
```

Problems Output Debug Console Terminal Ports

PS C:\Documentos\Codigos\Criptografia> & C:\Users\andre\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Documentos/Codigos/Criptografia/main.py

Traceback (most recent call last):

File "c:\Documentos\Codigos\Criptografia\main.py", line 29, in <module>

assert verificar_hmac(chave_legitima, original, etiqueta_original)

AssertionError

PS C:\Documentos\Codigos\Criptografia> & C:\Users\andre\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Documentos/Codigos/Criptografia/main.py

Teste 1 (SHA-256): Hash forjado pelo atacante com sucesso (sem chave).

Teste 2 (HMAC Legitimo): Servidor validou a etiqueta original.

Teste 3 (Ataque HMAC): Servidor REJEITOU a etiqueta criada com a chave do atacante.

PS C:\Documentos\Codigos\Criptografia>

2. Parece válida porque o SHA-256 é uma função de hash pública e não exige o uso de uma chave secreta, qualquer pessoa pode utilizá-lo. Ao interceptar a transferência e trocar o valor para 900, o atacante recalcula o hash usando a mesma função pública. Quem recebe a mensagem recalculada fará o hash localmente e verá que os valores são idênticos. Sem uma fonte de confiança extra (como uma chave compartilhada atrelando a origem), o receptor não tem como diferenciar

se o hash foi gerado pelo remetente original ou pelo atacante no meio do caminho.

3. Tabela de Propriedades

Propriedade	SHA-256	HMAC	Justificativa
Confidencialidade	Não	Não	Ambas as técnicas não cifram a mensagem (o texto permanece claro).
Integridade contra atacante	Não	Sim	No SHA-256 o atacante consegue recalcular o resumo. No HMAC, ele não possui a chave secreta para recalcular.
Autenticidade	Não	Sim	O HMAC usa chave compartilhada; se a etiqueta bate, a mensagem veio de quem tem a chave.
Não repúdio	Não	Não	O HMAC usa uma chave <i>compartilhada</i> entre as duas partes; portanto, não é possível provar a um terceiro qual dos dois enviou a mensagem.

4. Parágrafo de Entrega

O experimento demonstra na prática que funções de hash tradicionais, como o SHA-256, são insuficientes para garantir a integridade e autenticidade num ambiente de comunicação hostil, já que um atacante ativo pode adulterar a mensagem e simplesmente recalcular o resumo (hash) sem que o receptor consiga distinguir a manipulação. Ao implementarmos o HMAC-SHA-256, passamos a atrelar a validação do resumo a uma chave secreta que o atacante desconhece; desta forma, qualquer alteração na mensagem resultará em uma etiqueta que não poderá ser validada sem a chave, preservando a origem. Contudo, é fundamental observar que a confidencialidade continua inexistente, pois nenhuma

construção realiza a *cifração* do conteúdo, e o não repúdio não é atendido, dado que a chave do HMAC é uma posse compartilhada.

Código (Disponível também no github):

```
import hashlib
import hmac
import secrets

def sha256(mensagem):
    return hashlib.sha256(mensagem).digest()

def criar_hmac(chave, mensagem):
    # Produz um HMAC-SHA-256
    return hmac.new(chave, mensagem, hashlib.sha256).digest()

def verificar_hmac(chave, mensagem, etiqueta):
    # Recalcula a etiqueta e usa hmac.compare_digest para uma
    # comparação segura
    etiqueta_calculada = criar_hmac(chave, mensagem)
    return hmac.compare_digest(etiqueta_calculada, etiqueta)

original = b"destino=ana; valor $=100"
alterada = b"destino=ana; valor $=900"

# SHA-256 nao possui chave. O atacante consegue recalcular o resumo.
hash_original = sha256(original)
hash_forjado = sha256(alterada)
assert hash_forjado == sha256(alterada)
print("Teste 1 (SHA-256): Hash forjado pelo atacante com sucesso (sem
chave).")

# servidor e o remetente compartilham esta chave; o atacante nao.
chave_legitima = secrets.token_bytes(32)
chave_atacante = secrets.token_bytes(32)

etiqueta_original = criar_hmac(chave_legitima, original)
assert verificar_hmac(chave_legitima, original, etiqueta_original)
print("Teste 2 (HMAC Legitimo): Servidor validou a etiqueta original.")

# Crie uma etiqueta para a mensagem alterada com a chave do atacante
# e confirme que o servidor a rejeita.
etiqueta_atacante = criar_hmac(chave_atacante, alterada)
```

```
# O assert not valida que o servidor REJEITOU a verificação (retornou False)
assert not verificar_hmac(chave_legitima, alterada, etiqueta_atacante)
print("Teste 3 (Ataque HMAC): Servidor REJEITOU a etiqueta criada com a chave do atacante.")
```

Desafio de Fechamento

Situação escolhida: Aplicativo de Mensagens (WhatsApp).

Propriedades de segurança presentes no sistema:

1. **Confidencialidade:** Garantida pela cifração (como a criptografia de ponta a ponta), que impede que bisbilhoteiros, provedores de internet ou a própria empresa que fornece o aplicativo leiam o conteúdo da conversa enquanto ela atravessa a rede.
2. **Integridade:** Utilizada para detectar qualquer modificação não autorizada nos dados. Garante que um atacante ativo não consiga interceptar a comunicação e alterar a palavra "Sim" para "Não" sem que o aplicativo do destinatário detecte e rejeite a mensagem.
3. **Autenticidade:** Fornece evidência sobre a origem da mensagem. Permite que o seu aplicativo verifique e confirme que a mensagem realmente veio do dispositivo da pessoa com quem você acha que está conversando, e não de um invasor forjando o remetente.
4. **Não repúdio:** Com o uso das chaves criptográficas apropriadas atreladas a uma identidade, gera-se uma evidência que ajuda terceiros a atribuir a ação. Assim, um usuário não pode simplesmente negar ter enviado uma mensagem criminosa ou ameaçadora que saiu do seu dispositivo.

Limitação: A criptografia protege o dado em trânsito, mas não resolve a segurança no ponto de extremidade (no próprio dispositivo) após a decifração. Como o dado pode permanecer exposto na tela de uma pessoa autorizada, a criptografia não impede que o destinatário legítimo tire um *print* (captura de tela) da conversa e a vaze na internet.